



Proxy Pattern

1 Mục tiêu bài học

Sau bài học này, sinh viên có thể:

- Hiểu bản chất và mục đích của Proxy Pattern
- Phân biệt các loại Proxy: Virtual, Protection, Remote, Smart
- Áp dụng Proxy trong Order Management System (OMS)
- So sánh Proxy với Decorator và Adapter
- Triển khai Proxy Pattern bằng C#

2 Vấn đề

Trong hệ thống Order Management System, ta thường gặp:

- ◆ Vấn đề 1: Đối tượng nặng (Heavy Object) - Tải chi tiết đơn, PDF Invoice, Payment Gateway.
- ◆ Vấn đề 2: Kiểm soát quyền truy cập - Chỉ Admin hủy đơn, Kế toán xem báo cáo.
- ◆ Vấn đề 3: Truy cập hệ thống từ xa - Gọi API thanh toán, microservice inventory.

Việc khởi tạo trực tiếp gây: Tốn tài nguyên, chậm hệ thống, giảm performance.

3 Giải pháp & **4** Cấu trúc

3 Giải pháp: Proxy Pattern

Proxy cung cấp một đối tượng đại diện để kiểm soát truy cập đến một đối tượng khác.

Client không gọi trực tiếp đối tượng thật (RealSubject) → gọi thông qua Proxy.

4 Cấu trúc

ISubject – Interface chung

RealSubject – Đối tượng thật

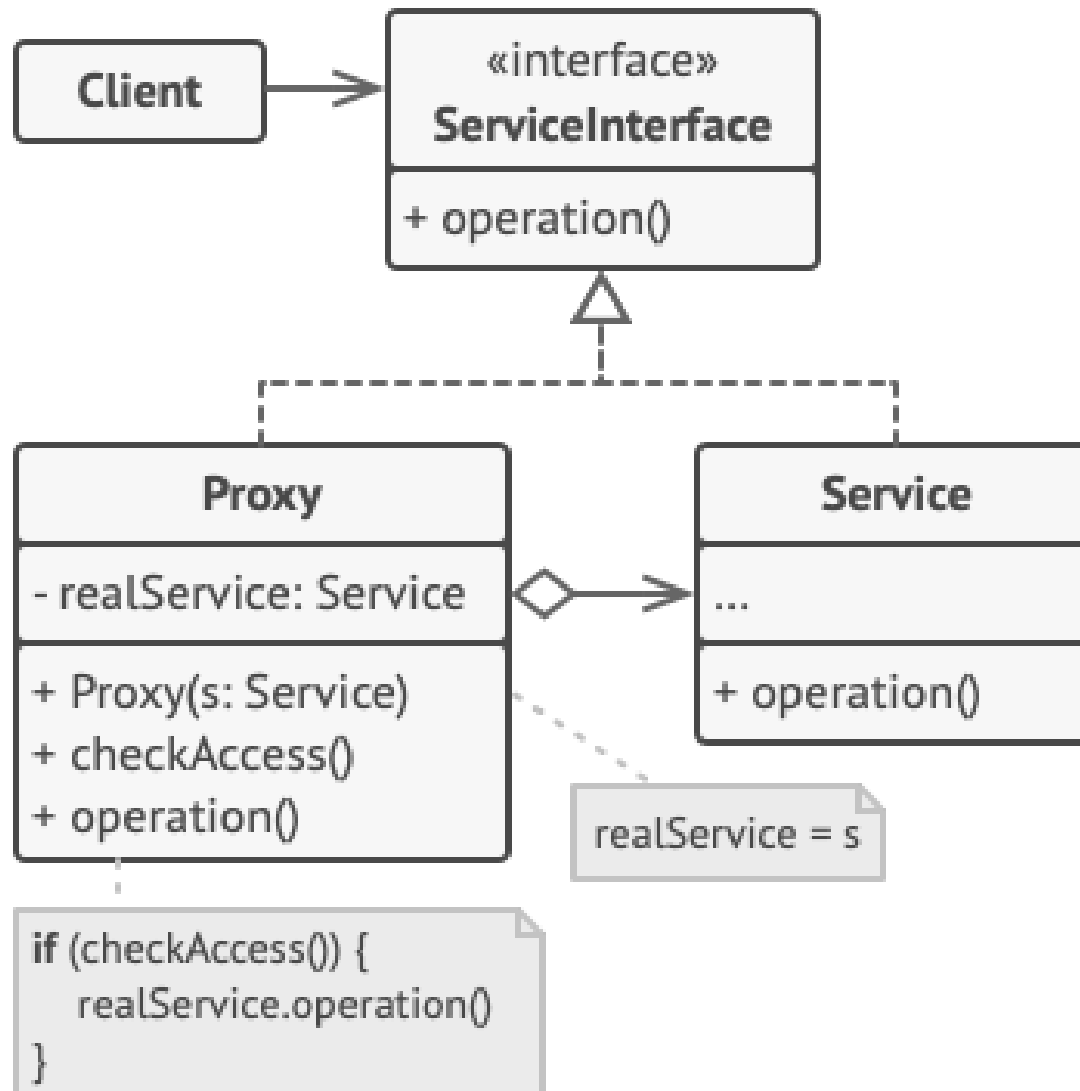
Proxy – Đại diện kiểm soát truy cập

Client – Sử dụng interface

Sơ đồ: Client → Proxy → RealSubject



Structure of Proxy Pattern



Ví dụ 1: Virtual Proxy (Lazy Loading)

Interface & Real Subject

```
public interface IOrderService { void DisplayOrderDetails(); }

public class RealOrderService : IOrderService {

    public RealOrderService(string orderId) { LoadHeavyData(); }

    private void LoadHeavyData() { Console.WriteLine("Loading heavy data..."); }

    public void DisplayOrderDetails() { /* Display logic */ }

}
```

Proxy Implementation

```
public class OrderServiceProxy : IOrderService {

    public void DisplayOrderDetails() {

        if (_realService == null) _realService = new RealOrderService(_orderId);

        _realService.DisplayOrderDetails();

    }

}
```

Ví dụ 2: Protection Proxy

Bài toán: Chỉ Admin được hủy đơn hàng.

Protection Proxy

```
public class OrderActionProxy : IOrderAction {  
    public void CancelOrder() {  
        if (_role != "ADMIN") {  
            Console.WriteLine("Access Denied: Only ADMIN can  
cancel orders.");  
            return;  
        }  
        _realAction.CancelOrder();  
    }  
}
```

Ví dụ 3: Smart Proxy & Ví dụ 4: Remote Proxy

Ví dụ 3: Smart Proxy (Logging + Performance)

Sử dụng Stopwatch để đo Execution time và Logging trước/sau khi gọi RealService.

Ví dụ 4: Remote Proxy (Payment Service)

```
public class PaymentServiceProxy : IPaymentService {  
    public void ProcessPayment(decimal amount) {  
        Console.WriteLine("Validating payment request...");  
        if (_realService == null) _realService = new RealPaymentService();  
        _realService.ProcessPayment(amount);  
    }  
}
```




So sánh Proxy với các Pattern khác

Pattern	Mục đích
Proxy	Kiểm soát truy cập
Decorator	Thêm chức năng
Adapter	Chuyển đổi interface

Khi nào nên dùng & Bài tập thực hành

 Khi nào nên dùng Proxy?

✓ Đối tượng nặng cần lazy loading | ✓ Cần kiểm soát quyền truy cập

✓ Làm việc với tài nguyên từ xa | ✓ Cần logging hoặc caching

 Bài tập thực hành

1. Tạo InvoiceServiceProxy chỉ load PDF khi Download.
2. Tạo ReportProxy chỉ cho phép role "ACCOUNTANT".
3. Thêm caching vào OrderServiceProxy.
4. Thiết kế Proxy cho Inventory API (giả lập retry 3 lần).



Kết luận

Proxy Pattern giúp: Tăng performance, Tăng bảo mật, Tách biệt trách nhiệm, Dễ mở rộng.

Trong Order Management System, Proxy rất hữu ích cho:

- Quản lý quyền
- Lazy loading dữ liệu lớn
- Giao tiếp microservices
- Logging và monitoring